

Pensamento Computacional

Ronierison Maciel

Agosto 2025



Estácio

Quem sou eu?



Nome: Roni

Formação: Mestre em Ciência da Computação

Ocupação: Pesquisador, Professor e Escovador de bits

Hobbies: Jogar cartas, ficar com a família

Interesses: Carros, DevSec, DS e ML

Email:

`ronierison.maciel@professores.estacio.br`

GitHub: <https://github.com/0x03c1>



Estácio

Conteúdo

- 1 Introdução ao Pensamento Computacional
- 2 Decomposição de Problemas
- 3 Abstração
- 4 Reconhecimento de Padrões
- 5 Algoritmos
- 6 PC em Economia Criativa, Negócios e Ciências Jurídicas
- 7 Introdução à Lógica de Programação
- 8 Scratch I (Fundamentos)
- 9 Scratch II (Estruturas de Controle)
- 10 Scratch III (Variáveis e Contadores)
- 11 Scratch IV (Projeto Guiado: Mini-Jogo)
- 12 Python I (Introdução ao Ambiente)
- 13 Python II (Operadores e Expressões)
- 14 Python III (Estruturas Condicionais)
- 15 Python IV (Estruturas de Repetição)
- 16 Python V (Listas e Estruturas de Dados Simples)
- 17 Python VI (Funções)



Estácio

- Entender o que é **Pensamento Computacional (PC)** e por que é útil.
- Diferenciar **PC** de **programação**.
- Conhecer os **quatro pilares**: decomposição, padrões, abstração e algoritmos.
- Vivenciar atividades **desplugadas** que ilustram algoritmos e precisão.



- Conceito de PC e *para que serve*.
- **PC** × **Programação**.
- Os 4 pilares do PC.
- Ciclo de resolução de problemas com PC.
- Atividades: *Algoritmo do Sanduíche* e *Robô Humano*.
- Quiz rápido e síntese.



O que é Pensamento Computacional?

- Maneira de **resolver problemas** de forma lógica e estruturada.
- Foco em **como pensar** sobre o problema antes de escrever código.
- Transfere-se para qualquer área: saúde, direito, administração, educação, etc.



Estácio

PC não é (apenas) programar

- **PC**: análise do problema, estratégia, passos, verificação.
- **Programar**: implementar a solução em uma linguagem (Python, etc.).
- Você pode praticar PC **sem computador** (atividades desplugadas).



Estácio

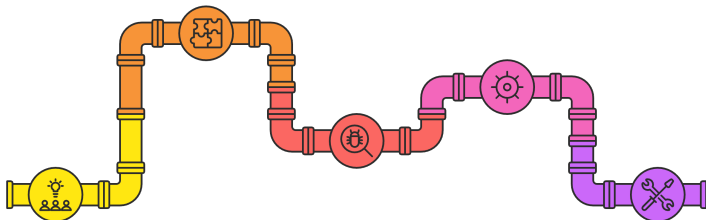
- **Decomposição:** dividir problemas complexos em partes menores.
- **Abstração:** focar no essencial e ignorar detalhes irrelevantes.
- **Reconhecimento de Padrões:** identificar similaridades e regularidades.
- **Algoritmos:** sequências **claras** e **finitas** de passos.



- Entender o problema e **definir o objetivo**.
- **Decomp**or em subproblemas.
- Buscar **padrões** e criar **abstrações**.
- Formular **algoritmo(s)** e **testar**.
- **Refinar** (depurar, melhorar, simplificar).



Processo de Resolução de Problemas com PC



01

Entender o problema e definir o objetivo

Identificar
claramente o que
precisa ser resolvido

02

Decompor em subproblemas

Dividir o problema em partes menores e mais gerenciáveis

03

Buscar padrões e criar abstrações

Observar
semelhanças e
eliminar detalhes
desnecessários

04

Formular algoritmos e testar

Criar passos lógicos para resolver o problema e verificar se funcionam

05

Refinar a solução

Melhorar, corrigir e simplificar o algoritmo ou abordagem



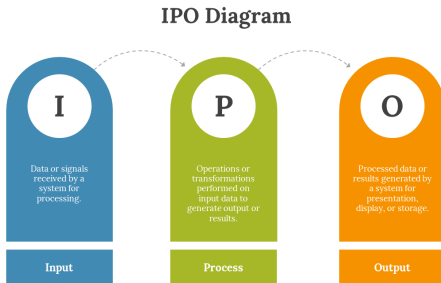
Estácio

- **Receita** de bolo: lista de ingredientes (*entradas*) e passos (*algoritmo*).
- **Viagem**: planejar rota, custos, tempo (*decomposição*).
- **Resumo** de texto: filtrar ideias principais (*abstração*).



Modelo E-P-S (Entrada, Processamento, Saída)

- **Entrada:** dados que entram no sistema.
- **Processamento:** regras/ algoritmo aplicadas aos dados.
- **Saída:** resultados gerados.



Estácio

Propriedades de um Algoritmo

- **Clareza:** cada passo é inequívoco.
- **Finitude:** termina em tempo finito.
- **Corretude:** resolve o problema proposto.
- **Generalidade:** funciona para vários casos do mesmo tipo.



Estácio

Pseudocódigo — Algoritmo do Sanduíche

```
1 # Objetivo: descrever passos claros e finitos
2 # para montar um sanduíche de forma "executável".
3
4 passos = [
5     "Lavar as mãos",
6     "Pegar 2 fatias de pão",
7     "Passar recheio (ex.: queijo) na primeira fatia",
8     "Colocar alface e tomate (opcional)",
9     "Fechar com a segunda fatia",
10    "Cortar ao meio",
11    "Servir no prato"
12 ]
13
14 for p in passos:
15     print(p)
```



Estácio

Precisão importa (e muito!)

- Instruções **vagas** geram resultados **inesperados**.
- Ex.: “colocar queijo” — quanto? onde? em qual lado?
- PC exige **detalhamento suficiente** para execução correta.



Estácio

- Em duplas, escrevam **passos mínimos** para montar um sanduíche.
- Troquem com outra dupla e **sigam à risca** o algoritmo recebido.
- Observem: onde faltou clareza? O que precisa ser **refinado**?



- Um colega é o **robô** que entende poucos comandos:
 - andar,
 - virar,
 - pegar.
- A turma programa o robô para executar uma tarefa simples.
- Objetivo: perceber a importância de
 - **sequência** e
 - **condições**.



Teste seus conhecimentos!



Estácio

Hello, Mundo! (para demonstração)

```
1 # Demonstração de execução e saída (não é foco ainda).  
2 print("Olá, mundo!")  
3 nome = input("Seu nome: ")  
4 print(f"Bem-vindo(a), {nome}!")
```



Estácio

- PC = **forma de pensar** problemas.
- Quatro pilares como **ferramentas mentais**.
- Atividades mostraram **precisão** e **sequência**.
- Próxima aula: **Decomposição** a fundo.



- Entender **o que é** e **por que** usar decomposição.
- Praticar **técnicas** (top-down, refinamento sucessivo).
- Exercitar **critérios de boa decomposição** (coesão, baixo acoplamento).
- Aplicar em casos reais e preparar terreno para padrões.



- Conceitos e benefícios.
- Estratégias e armadilhas comuns.
- Estudos de caso (cotidiano e computação).
- Prática e exercício em grupo.



Decomposição: Conceito e Benefícios



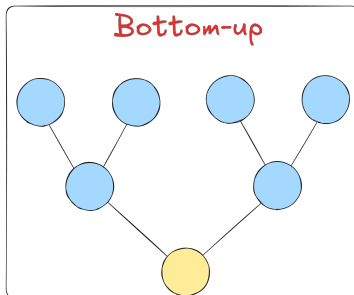
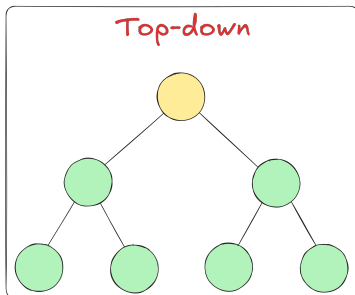
- Benefícios:
 - Reduz complexidade cognitiva.
 - Permite **dividir e conquistar** em equipe.
 - Facilita **testes e reuso**.



Estácio

Estratégias Comuns

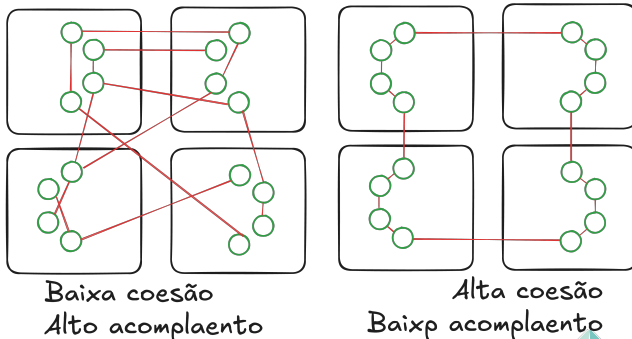
- **Top-Down:** do geral para o específico (*refinamento sucessivo*).
- **Bottom-Up:** partir de componentes prontos/conhecidos.
- **Híbrida:** combina as duas conforme o contexto.



Estácio

Cr terios de Boa Decomposi  o

- **Coes o**: cada parte resolve uma ** nica responsabilidade**.
- **Baixo acoplamento**: partes **pouco dependentes** entre si.
- **Interfaces claras**: entradas/s idas bem definidas.



Est cio

Exemplo 1 — Organizar uma Festa

- Subtarefas:
 - Convidados e convites.
 - Local e logística.
 - Comidas e bebidas.
 - Música/entretenimento.
 - Orçamento e compras.
- Cada subtarefa tem **entradas, processos, saídas**.



Estácio

Exemplo 2 — Calculadora de Média (Computação)

- Subproblemas:
 - Ler notas (*entrada*).
 - Calcular média (*processamento*).
 - Exibir resultado e situação (*saída*).
- Partes podem ser **funções** independentes.



Estácio

```
1 def ler_notas():
2     # abstrai a origem das notas (teclado, arquivo, etc.)
3     notas = []
4     qtd = int(input("Quantas notas? "))
5     for _ in range(qtd):
6         notas.append(float(input("Nota: ")))
7     return notas
8
9 def calcular_media(notas):
10     return sum(notas) / len(notas) if notas else 0.0
11
12 def exibir_resultado(media):
13     print(f"Média: {media:.2f}")
14     if media >= 7.0:
15         print("Situação: Aprovado(a)")
16     elif media >= 5.0:
17         print("Situação: Recuperação")
18     else:
19         print("Situação: Reprovado(a)")
20
21 def main():
22     notas = ler_notas()
```



```
23     media = calcular_media(notas)
24     exibir_resultado(media)
25
26 if __name__ == "__main__":
27     main()
```



Armadilhas Comuns

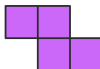
Dependências Circulares

As partes dependem uma da outra



Partes Gigantes

Granularidade de decomposição insuficiente



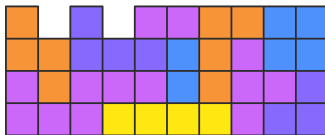
Interfaces Confusas

Entradas e saídas mal definidas



Partes Minúsculas Demais

Sobrecarga sem ganho real



Estácio

Checklist de Decomposição

- Cada parte tem **objetivo claro**?
- Entradas/saídas estão **explícitas**?
- Consigo **testar** esta parte isoladamente?
- Esta parte será **reutilizável** em outros contextos?



Estácio

- Problema: **reduzir lixo na IES.**

- ① Definir objetivo e métricas (ex.: % de redução/mês).
- ② Decompor em frentes: educação, coleta, parceria, comunicação.
- ③ Para cada frente: entradas, processos, saídas.
- ④ Integrar plano e priorizar ações.



- Em trios/quartetos, criem um **mapa de decomposição**.
- Entrega:
 - Lista de subproblemas com 1–2 linhas cada.
 - Entradas/saídas de cada subproblema.
 - Critérios de sucesso (curtos e mensuráveis).
- **Tempo:** 20–25 minutos + 5 min de compartilhamento.



Mini-Caso — Decompor um App de *To-Do*

- Subproblemas:
 - Cadastro de tarefas.
 - Listagem e filtros.
 - Marcar como concluída.
- Critérios: coesão por responsabilidade e baixo acoplamento.



Estácio

```
1 # Este pseudocódigo descreve a "foto" da solução,  
2 # sem entrar em detalhes internos de cada parte.  
3  
4 # Módulo Entrada  
5 def coletar_dados():  
6     # ler dados de uma fonte (teclado/arquivo/etc.)  
7     return dados  
8  
9 # Módulo Processamento  
10 def processar(dados):  
11     # transformar dados em informação útil (regras/  
12     algoritmos)  
13     return resultado  
14  
15 # Módulo Saída  
16 def apresentar(resultado):  
17     # exibir/registrar o resultado  
18     pass  
19  
20 def pipeline():  
21     dados = coletar_dados()  
22     resultado = processar(dados)
```

```
apresentar(resultado)
```

**Estácio**

- Decompor reduz complexidade e melhora colaboração.
- Critérios: **coesão, baixo acoplamento, interfaces claras.**
- Próxima aula: **Abstração** (reaproveitar soluções).



Teste seus conhecimentos!



Estácio

- Compreender o conceito de **abstração** no Pensamento Computacional.
- Saber diferenciar o **essencial** dos detalhes **irrelevantes**.
- **Aplicar** abstração em problemas do **cotidiano** e em **algoritmos**.
- Vivenciar exercícios práticos de **filtragem** de **informações**.



- Definição e importância da abstração.
- Exemplos práticos do dia a dia.
- Abstração em programação.
- Atividades: resumo de texto, jogo dos 20 perguntas.
- Exercício prático com Scratch.
- Síntese e discussão.



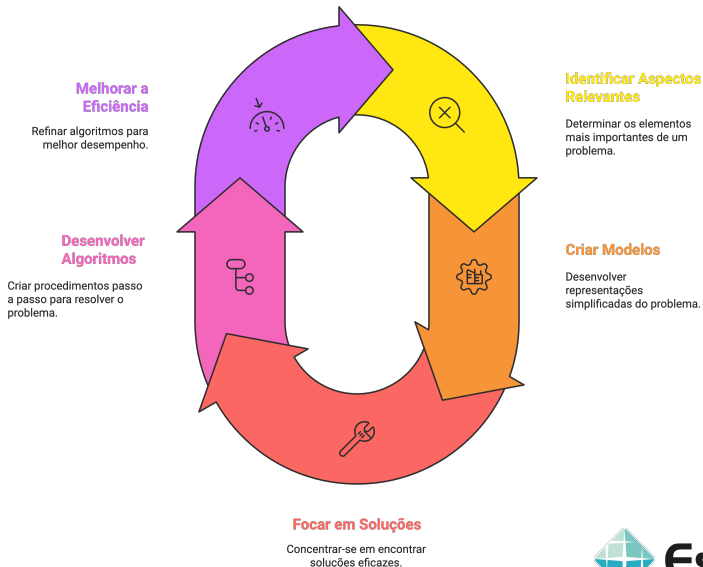
O que é Abstração?

- 2º pilar do Pensamento Computacional.
- **Filtrar o que é relevante**, ignorando detalhes supérfluos.
- Criar uma **representação simplificada** de um problema.
- Permite lidar com complexidade de forma eficiente.



Estácio

Ciclo de Abstração em Processamento Computacional



Estácio

Exemplos de Abstração no Cotidiano

- **Mapa:** mostra ruas principais, não cada árvore ou detalhe.
- **Resumo de texto:** extrai apenas ideias centrais.
- **Receita:** não descreve cada movimento da mão, apenas os passos principais.



Estácio

Benefícios da Abstração

- **Reduz** a complexidade percebida.
- Foco nos aspectos mais **importantes**.
- Facilita **comunicação** entre pessoas.
- Cria modelos **reutilizáveis**.



Estácio

- Funções encapsulam detalhes — o usuário só chama a função.
- Classes e objetos escondem implementações internas.
- Exemplo: usamos `print()` sem saber todos os detalhes de baixo nível.



Exemplo em Python — Função Abstrata

```
1 # Função abstrata: esconder detalhes de cálculo
2 def calcular_media(notas):
3     return sum(notas) / len(notas)
4
5 # Quem usa a função não precisa saber como a média é
   calculada
6 valores = [8.0, 7.5, 9.0]
7 print("Média:", calcular_media(valores))
```



Estácio

Atividade — Jogo dos 20 Perguntas

- Um aluno pensa em um **objeto/pessoa**.
- Os colegas podem perguntar apenas "**Sim/Não**".
- **Estratégia:** usar abstrações para eliminar grupos de possibilidades.
- **Objetivo:** mostrar como filtrar informações reduz complexidade.
 - Possíveis perguntas:
 - É um objeto vivo? → **Não**
 - É um objeto tecnológico? → **Sim**
 - Usamos no dia a dia? → **Sim**
 - Serve para fazer ligações? → **Sim**
 - O aluno pensou em um celular.



Estácio

- **Abstração** = foco no essencial, ignorando detalhes supérfluos.
- Presente no cotidiano (**mapas, resumos, modelos**).
- Em programação: **funções, classes, APIs**.
- Próxima aula: **Reconhecimento de Padrões**.



Teste seus conhecimentos!



Estácio

- Entender o que é **reconhecimento de padrões**.
- Identificar padrões em problemas do cotidiano e em algoritmos.
- Explorar como padrões aceleram soluções e evitam retrabalho.
- Vivenciar atividades práticas (sequências, jogos e desafios).



- **Conceito** e importância de **padrões**.
- Exemplos do cotidiano.
- **Padrões** em programação e **algoritmos**.
- Atividades práticas: sequências e jogo puzzle.
- Discussão e síntese.



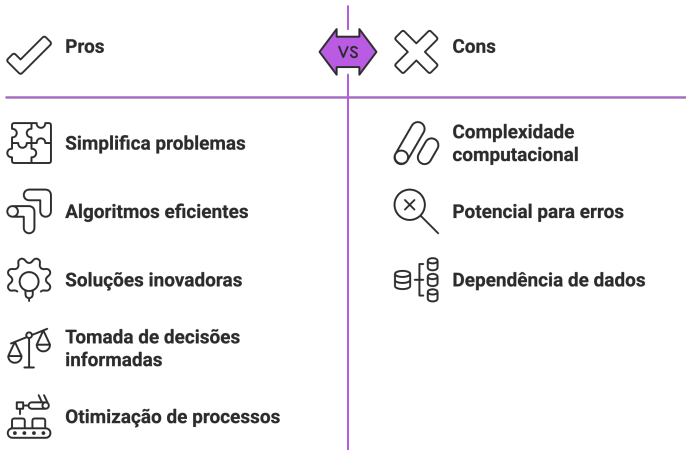
O que é Reconhecimento de Padrões?

- 3º pilar do Pensamento Computacional.
- Identificar **similaridades, repetições ou tendências**.
- Usado para **prever, generalizar e reutilizar soluções**.
- Exemplo: perceber que todo problema de cálculo de média segue o mesmo padrão de passos.



Estácio

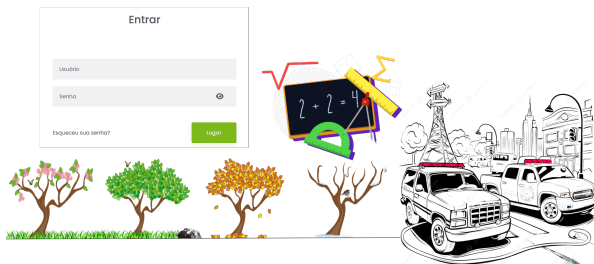
Pros e Contra



Estácio

Exemplos do Cotidiano

- **Matemática:** sequências numéricas (2, 4, 6, 8...).
- **Natureza:** ciclos das estações do ano.
- **Trânsito:** semáforo com sequência previsível (verde–amarelo–vermelho).
- **Tecnologia:** senha com formato padrão (ex: 3 letras + 3 números).



Estácio

Benefícios de Identificar Padrões

- **Reduz esforço**: solução de um caso pode ser reaplicada em outros.
- Aumenta **velocidade de resolução**.
- Facilita **abstração** futura.
- Permite **prever** resultados antes de ocorrerem.



Estácio

- **Estruturas comuns:** leitura de dados, processamento, saída.
- Laços de repetição representam **padrões de execução**.
- **Funções:** encapsulam soluções reutilizáveis.
- **Exemplo:** padrão "ler, processar, imprimir" em vários programas.



Exemplo em Python — Padrão Reutilizável

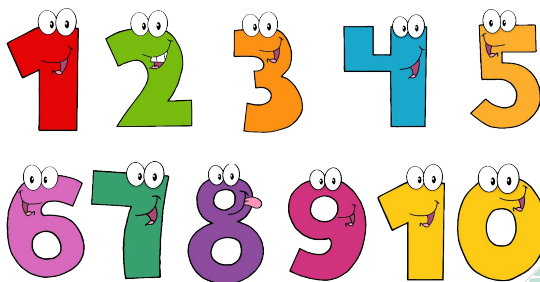
```
1 # Padrão: Ler lista de números, processar, mostrar
2 def calcular_media(lista):
3     return sum(lista) / len(lista)
4
5 dados = [7.0, 8.5, 9.0, 6.5]
6 media = calcular_media(dados)
7 print("Média:", media)
8
9 # Esse padrão se repete em várias aplicações!
```



Estácio

Atividade — Sequências Numéricas

- Qual o próximo número?
 - 2, 4, 6, 8, ?
 - 1, 2, 4, 8, 16, ?
 - 5, 10, 20, 40, ?
- O que muda em cada padrão?
- Como descrever em forma de regra?



Estácio

- Acessar jogo/puzzle que exige sequências de passos.
- Observar como repetições geram **padrões de movimento**.
- Comparar soluções: quem percebeu o padrão usou menos instruções!
- Acessa o código QR abaixo:



- Dividir em grupos de 3–4 alunos.
- Cada grupo deve criar 2 exemplos de padrões:
 - Um do cotidiano (**vida real**).
 - Um ligado à **tecnologia/programação**.
- Compartilhar com a turma o padrão.



- Reconhecer padrões = economizar tempo e esforço.
- Cotidiano e programação estão cheios de padrões.
- Atividades mostraram como padrões podem **encurtar soluções**.
- Próxima aula: **Algoritmos**.



- Entender o conceito de **algoritmo**.
- Reconhecer algoritmos no cotidiano.
- Compreender propriedades fundamentais: clareza, finitude e ordem.
- Representar algoritmos em **linguagem natural**, **fluxogramas**, **pseudocódigo** e código Python simples.



- Definição de algoritmo.
- Propriedades essenciais.
- Exemplos do dia a dia.
- Representações: natural, fluxograma, pseudocódigo.
- Demonstração em Python.
- Atividade prática: criar e testar algoritmos.



O que é um Algoritmo?

- Conjunto **finito e ordenado** de instruções.
- Resolve um problema ou executa uma tarefa.
- Entrada $\rightarrow$ Processamento $\rightarrow$ Saída.



Estácio

Propriedades de um Algoritmo

- **Clareza:** passos não ambíguos.
- **Finitude:** termina em algum momento.
- **Corretude:** chega à solução esperada.
- **Eficiência:** resolve em tempo e esforço razoáveis.



Estácio

Exemplos do Cotidiano

- Receita de bolo.
- Instruções de montagem de um móvel.
- Regras de um jogo (ex.: xadrez).
- Procedimento para resolver equação matemática.



Estácio

- **Linguagem natural:** passos em texto.
- **Fluxogramas:** símbolos gráficos conectados.
- **Pseudocódigo:** estrutura simplificada, próxima da programação.
- **Código:** implementação em uma linguagem (ex.: Python).



Exemplo em Linguagem Natural

- Algoritmo para verificar se um número é par:
 - 1 Ler um número.
 - 2 Dividir o número por 2.
 - 3 Se o resto for 0, dizer que é par.
 - 4 Caso contrário, dizer que é ímpar.



Estácio

Exemplo em Pseudocódigo

```
1 # Verificar se um número é par
2 LEIA numero
3 SE numero % 2 == 0 ENTAO
4     ESCREVA "Número par"
5 SENAO
6     ESCREVA "Número ímpar"
7 FIM
```



Estácio

Exemplo em Python

```
1 numero = int(input("Digite um número: "))
2
3 if numero % 2 == 0:
4     print("Número par")
5 else:
6     print("Número ímpar")
```



Estácio

- Em duplas, escrevam um algoritmo que descreva o funcionamento de um semáforo:
 - **Verde:** carros andam, pedestres param.
 - **Amarelo:** alerta de atenção.
 - **Vermelho:** carros param, pedestres andam.
- Representem em linguagem natural, fluxograma e pseudocódigo.



- Problema: “Calcular a média de 3 notas e mostrar se aprovado ou reprovado”.
- Cada grupo cria:
 - 1 Passos em linguagem natural.
 - 2 Pseudocódigo.
 - 3 Código Python (opcional).
- Apresentar a solução para a turma.



- **Algoritmos** = sequência de passos lógicos e finitos.
- Representações múltiplas: natural, fluxograma, pseudocódigo, código.
- Atividades mostraram clareza e precisão necessárias.
- Próxima aula: **Introdução à Lógica de Programação.**

Nota: Aplique em uma dificuldade que tenha ou em uma situação de seu cotidiano os 4 pilares do pensamento computacional e observe como eles podem facilitar ou acelerar sua resolução.



Agora que você já sabe o que é pensamento computacional, vamos entender como ele se aplica às mais variadas áreas do conhecimento e como ele vem transformando o mundo a nosso redor, por meio da seguinte linha cronológica:



- Compreender a ideia de **lógica de programação**.
- Entender conceitos básicos: sequência, decisão e repetição.
- Explorar o raciocínio algorítmico aplicado a problemas.
- Vivenciar atividades desplugadas e primeiros contatos com pseudocódigo.

NOTA: Usar o pensamento computacional não é saber utilizar as ferramentas. É saber usar a tecnologia para criar novas ferramentas.



- Definição de lógica de programação.
- Três estruturas fundamentais.
- Exemplos do cotidiano.
- Atividade desplugada: “Robô Humano”.
- Demonstrações em pseudocódigo e Python.
- Exercícios em grupo.



O que é Lógica de Programação?

- Conjunto de regras e raciocínios usados para resolver problemas.
- Base da programação: pensar no **como fazer** antes do código.
- Utiliza estruturas simples: sequência, decisão e repetição.



Estácio

- **Sequência:** passos executados em ordem.
- **Decisão (condicional):** executar algo se uma condição for verdadeira.
- **Repetição (loop):** repetir uma ação enquanto houver necessidade.



Exemplo Cotidiano — Sequência

- Escovar os dentes:
 - 1 Pegar escova.
 - 2 Colocar pasta.
 - 3 Escovar.
 - 4 Enxaguar.
- Ordem importa: se inverter, perde o sentido.



Estácio

- “Se estiver chovendo, então levar guarda-chuva.”
- Estrutura:
 - **Condição:** está chovendo?
 - **Ação:** levar guarda-chuva.
 - **Senão:** não levar.



Exemplo Cotidiano — Repetição

- “Encha o copo de água até ficar cheio.”
- Enquanto não estiver cheio, continue enchendo.
- Estrutura típica de um **loop while**.



Estácio

Demonstração em Pseudocódigo

```
1 # Estruturas básicas em pseudocódigo
2
3 # Sequência
4 LEIA numero
5 ESCREVA numero * 2
6
7 # Decisão
8 SE numero > 0 ENTAO
9     ESCREVA "Positivo"
10 SENAO
11     ESCREVA "Negativo ou zero"
12
13 # Repetição
14 ENQUANTO numero < 10 FACA
15     numero = numero + 1
16     ESCREVA numero
17 FIMENQUANTO
```



Estácio

Exemplo em Python

```
1 # Sequência
2 nome = input("Digite seu nome: ")
3 print("Ola,", nome)
4
5 # Decisão
6 idade = int(input("Digite sua idade: "))
7 if idade >= 18:
8     print("Maior de idade")
9 else:
10    print("Menor de idade")
11
12 # Repetição
13 contador = 1
14 while contador <= 5:
15     print("Contagem:", contador)
16     contador += 1
```



Estácio

Atividade Desplugada — Robô Humano

- Um aluno é o “robô” e só entende comandos básicos: andar, virar, pegar.
- A turma cria uma sequência de instruções para guiá-lo em uma tarefa.
- Objetivo: perceber importância de clareza, decisão e repetição.



Estácio

- Problema: Criar algoritmo para calcular o quadrado de um número.
- Em duplas:
 - 1 Escrevam em linguagem natural.
 - 2 Transformem em pseudocódigo.
 - 3 Implementem em Python.



- Lógica de programação é a **base** do raciocínio **computacional**.
- Estruturas essenciais: **sequência**, **decisão** e **repetição**.
- Atividades mostraram como pensar em algoritmos antes de codificar.
- Próxima aula: **Scratch I — Primeiros Passos**.



- Conhecer o ambiente do **Scratch**.
- Compreender como **blocos** representam comandos **lógicos**.
- Construir o **primeiro** projeto **interativo** simples.
- Relacionar conceitos de **lógica** de programação com **programação** visual.



- Introdução ao Scratch.
- Interface: palco, blocos, atores.
- Categorias principais de blocos.
- Demonstração: programa de boas-vindas.
- Atividade prática em laboratório.



O que é Scratch?

- **Plataforma** de programação **visual** em blocos.
- Voltada para **ensino** e experimentação **criativa**.
- Permite criar **animações**, **jogos** e **histórias interativas**.
- Desenvolvido pelo **MIT Media Lab**.



Estácio

Interface do Scratch

- **Palco:** onde a ação acontece.
- **Atores (sprites):** personagens/objetos que executam ações.
- **Blocos:** instruções de movimento, controle, aparência, som, etc.
- **Área de scripts:** onde arrastamos e organizamos blocos.



Estácio

- **Movimento:** andar, girar, deslizar.
- **Aparência:** falar, mudar cor, esconder.
- **Som:** tocar áudio ou música.
- **Controle:** repetições, condições.
- **Sensores:** detectar teclas, mouse, colisões.
- **Variáveis:** guardar valores.



- Quando a bandeira verde for clicada:
 - 1 O ator diz “Olá, mundo!”.
 - 2 Em seguida pergunta: “Qual é o seu nome?”.
 - 3 Responde com “Bem-vindo(a), [nome]!”.



- Em laboratório, cada aluno deve criar:
 - 1 Um sprite que fala “Olá!” ao clicar na bandeira.
 - 2 Depois se move 10 passos.
 - 3 Repete a fala 3 vezes.
- Compartilhar com um colega e testar.



- Criar um mini-projeto colaborativo:
 - Dois personagens conversam entre si.
 - Cada fala aparece em sequência (ordem lógica).
 - Um deles realiza um movimento após a fala.
- Objetivo: aplicar conceitos de **sequência** e **interação**.



- Scratch = programação visual por blocos.
- Blocos equivalem a comandos lógicos.
- Criamos nosso primeiro programa interativo.
- Próxima aula: **Scratch II — Estruturas de Controle.**



- Compreender as estruturas de **controle** no Scratch.
- Utilizar blocos de decisão e repetição em projetos.
- Relacionar estruturas de controle visuais com lógica de programação.
- Desenvolver projetos interativos com condições e loops.



- Revisão: blocos básicos e sequência.
- Estruturas de controle no Scratch.
- Demonstração: condicional simples.
- Demonstração: laço de repetição.
- Atividade prática em laboratório.
- Exercício em grupo: mini-jogo.



- **Decisão (if/else):** executa blocos se condição for verdadeira.
- **Repetição (loops):** repete instruções múltiplas vezes.
- **Eventos:** ativam scripts quando algo acontece.
- Fundamentais para criar comportamento dinâmico.



Exemplo — Decisão no Scratch

- Problema: “Se a tecla seta para cima for pressionada, o ator sobe.”
- Blocos:
 - 1 Quando tecla “↑” pressionada.
 - 2 Mover 10 passos para cima.



Estácio

Exemplo — Repetição no Scratch

- Problema: “Mover personagem 10 passos 5 vezes.”
- Blocos:
 - 1 Laço “repita 5 vezes”.
 - 2 Dentro do laço: mover 10 passos.



Estácio

- Construir programa: “Andar para frente até tocar na borda”.
- Blocos:
 - 1 Enquanto “não tocar na borda”.
 - 2 Mover 10 passos.
- Relacionar com o conceito de **loop while**.



- Criar jogo simples em que:
 - 1 Personagem anda com setas.
 - 2 Se encostar na cor da parede, volta ao início.
 - 3 Se alcançar a cor de destino, mostra mensagem “Você venceu!”.
- Tempo de laboratório: 25 minutos.



- Em duplas ou trios, criem um mini-jogo:
 - Personagem deve coletar um objeto.
 - Se encostar no objeto, som é tocado + pontos somados.
 - Repetir até atingir 5 pontos.
- Cada grupo apresenta a lógica usada.



- Estruturas de controle = **condições, repetições e eventos**.
- Permitem comportamento dinâmico e interatividade.
- Aplicadas em jogos, simulações e histórias.
- Próxima aula: **Scratch III — Variáveis e Contadores**.



- Compreender o conceito de **variável** no Scratch.
- Entender o papel de variáveis como "caixas de memória".
- Utilizar variáveis para armazenar valores e controlar ações.
- Criar programas que usam contadores e pontuações.



- O que são variáveis?
- Exemplos do cotidiano.
- Criando variáveis no Scratch.
- Demonstração: contador de cliques.
- Atividade prática: placar de jogo.
- Exercício em grupo.



O que são Variáveis?

- Espaços na memória que guardam valores.
- Podem mudar durante a execução do programa.
- Exemplo do cotidiano: uma calculadora que guarda o resultado.
- No Scratch: blocos da categoria “**Variáveis**”.



Estácio

Exemplos de Variáveis no Cotidiano

- **Placar** em um jogo de futebol.
- **Saldo** de uma conta bancária.
- **Nível** de energia em um videogame.
- **Temperatura** registrada em um termômetro.



Estácio

Criando Variáveis no Scratch

- Passos:

- 1 Clique na aba “**Variáveis**”.
- 2 Selecione “**Criar uma variável**”.
- 3 Defina um nome (ex.: “Pontos”, “Vidas”).
- 4 Use blocos “**mude para**” e “**some com**”.



Estácio

Demonstração — Contador de Cliques

- Objetivo: contar quantas vezes o usuário clica na bandeira verde.
- Blocos:
 - 1 Quando bandeira verde clicada $\rightarrow$ variável “cliques” = 0.
 - 2 Quando sprite clicado $\rightarrow$ some 1 a “cliques”.
 - 3 Mostrar variável “cliques” na tela.



Estácio

- Criar jogo em que:
 - 1 Personagem deve encostar em um objeto.
 - 2 Cada vez que encosta → soma 1 ponto.
 - 3 Mostrar pontuação na tela.
 - 4 Exibir mensagem “Você venceu!” quando atingir 10 pontos.
- Duração: 20 minutos.



Exercício em Grupo — Vidas do Personagem

- Em duplas ou trios, criar sistema de vidas:
 - Variável “Vidas” começa em 3.
 - Cada vez que personagem toca no obstáculo, perde 1 vida.
 - Se “Vidas” = 0, exibir mensagem de derrota.
- Apresentar solução à turma.



Estácio

- Variáveis = valores que podem ser armazenados e alterados.
- No Scratch, usadas para pontos, vidas, contadores etc.
- Atividades mostraram como variáveis dão “memória” ao programa.
- Próxima aula: **Scratch IV — Projeto Guiado (Mini-Jogo)**.



- Consolidar conceitos de blocos, controle e variáveis no Scratch.
- Construir um **mini-jogo interativo** guiado.
- Praticar colaboração e criatividade no desenvolvimento.
- Preparar terreno para projetos autorais nas próximas aulas.



- Revisão: sequência, decisão, repetição e variáveis.
- Proposta do mini-jogo.
- Demonstração passo a passo.
- Atividade prática em laboratório.
- Apresentação dos jogos criados.



- **Sequência:** ordem dos comandos.
- **Controle:** decisões e loops.
- **Variáveis:** armazenar pontos, vidas, contadores.
- **Eventos:** iniciar scripts com cliques ou teclas.



- Jogo “**Pegue a Estrela**”:
 - 1 Personagem principal se move com setas.
 - 2 Deve encostar em um objeto (estrela).
 - 3 Cada vez que encosta → ganha 1 ponto.
 - 4 Ao atingir 10 pontos → mensagem de vitória.



- Criar personagens:
 - Jogador (sprite principal).
 - Estrela (objeto coletável).
- Adicionar controles de movimento para o jogador:
 - ① Teclas de seta movem o personagem.



- Criar variável “Pontos”.
- Programar a estrela:
 - 1 Quando encostada → adicionar 1 ponto.
 - 2 Mover para posição aleatória.



- Verificar vitória:
 - ① Se pontos = 10 $\rightarrow$ mostrar mensagem “Você venceu!”.
- Opcional:
 - Adicionar tempo limite.
 - Inserir sons de coleta.



- Cada aluno implementa o mini-jogo.
- Sugestões de personalização:
 - Alterar sprites.
 - Mudar cenário.
 - Adicionar efeitos sonoros.
- Duração: 30 minutos.



- Construímos um mini-jogo completo no Scratch.
- Reforçamos conceitos de controle e variáveis.
- Estimulamos criatividade e colaboração.
- Próxima aula: **Python I — Introdução ao Ambiente.**



- Conhecer o que é **Python** e por que usá-lo.
- Aprender a configurar e acessar o ambiente de programação.
- Entender a estrutura básica de um programa em Python.
- Executar os primeiros comandos no interpretador.



- O que é Python e suas aplicações.
- Instalação e ambientes disponíveis.
- O interpretador (REPL) e editores online.
- Primeiros comandos: entrada, saída e operações.
- Atividade prática no laboratório.



Por que Python?

- Linguagem de fácil leitura e escrita.
- Usada em ciência de dados, IA, web, automação, etc.
- Comunidade grande e muitos recursos gratuitos.
- Ideal para iniciantes.



Estácio

- Instalação local (Python.org).
- IDEs: IDLE, VSCode, PyCharm.
- Plataformas online:
 - Replit (<https://replit.com>)
 - Cliprun (<https://cliprun.com/>)
 - Online Python (<https://www.online-python.com/>)
 - Python IDE (<https://www.pythonide.online/>)
 - Google Colab (<https://colab.research.google.com>)
 - Jupyter Notebook <https://jupyter.org>)



O Interpretador Python (REPL)

- REPL = Read, Evaluate, Print, Loop.
- Permite testar instruções de forma interativa.
- Ótimo para experimentos rápidos.



Estácio

Primeiros Comandos

```
1 # Exibir uma mensagem
2 print("Olá, mundo!")
3
4 # Operações matemáticas
5 print(2 + 3)
6 print(10 - 4)
7 print(6 * 7)
8 print(8 / 2)
9
10 # Variáveis simples
11 nome = "Roni"
12 idade = 30
13 print("Nome:", nome)
14 print("Idade:", idade)
```



Estácio

```
1 # Lendo dados do usuário
2 nome = input("Digite seu nome: ")
3 print("Bem-vindo,", nome)
4
5 numero = int(input("Digite um número: "))
6 print("O dobro é:", numero * 2)
```



- Cada aluno deve:
 - 1 Escrever um programa que pede o nome e a idade.
 - 2 Imprimir a mensagem: “Olá [nome], você tem [idade] anos!”.
 - 3 Calcular quantos anos terá daqui a 5 anos.
- Compartilhar com colega e rodar no computador dele.



- Em duplas, criar programa que:
 - 1 Pergunta o nome de duas pessoas.
 - 2 Pergunta a idade de cada uma.
 - 3 Mostra quem é mais velho.
- Apresentar a solução para a turma.



- Conhecemos o ambiente Python e seus usos.
- Executamos primeiros comandos no REPL.
- Aprendemos sobre entrada, saída e variáveis simples.
- Próxima aula: **Python II — Operadores e Expressões.**



- Entender os tipos de operadores em Python.
- Aplicar operadores aritméticos, relacionais e lógicos.
- Explorar expressões que combinam operadores e variáveis.
- Praticar com exemplos e exercícios em laboratório.



- Revisão de variáveis e entrada/saída.
- Operadores aritméticos.
- Operadores relacionais.
- Operadores lógicos.
- Expressões combinadas.
- Atividades práticas.



Operadores Aritméticos

- Soma: +
- Subtração: -
- Multiplicação: *
- Divisão: /
- Divisão inteira: //
- Resto (módulo): %
- Exponenciação: **



Estácio

Exemplo — Aritméticos

```
1 a = 10
2 b = 3
3
4 print("Soma:", a + b)
5 print("Subtração:", a - b)
6 print("Multiplicação:", a * b)
7 print("Divisão:", a / b)
8 print("Divisão inteira:", a // b)
9 print("Resto:", a % b)
10 print("Potência:", a ** b)
```



Estácio

Operadores Relacionais

- Igualdade: `==`
- Diferença: `!=`
- Maior que: `>`
- Menor que: `<`
- Maior ou igual: `>=`
- Menor ou igual: `<=`



Estácio

Exemplo — Relacionais

```
1 x = 5
2 y = 8
3
4 print(x == y)    # False
5 print(x != y)    # True
6 print(x > y)     # False
7 print(x < y)     # True
8 print(x >= 5)    # True
9 print(y <= 10)   # True
```



Estácio

- `and` $\rightarrow$ verdadeiro se ambas condições forem verdadeiras.
- `or` $\rightarrow$ verdadeiro se pelo menos uma condição for verdadeira.
- `not` $\rightarrow$ inverte o valor lógico.



Exemplo — Lógicos

```
1 idade = 20
2 tem_carteira = True
3
4 # Condição composta
5 pode_dirigir = (idade >= 18) and tem_carteira
6 print("Pode dirigir?", pode_dirigir)
7
8 # Usando OR
9 estudante = False
10 desconto = estudante or (idade < 12)
11 print("Tem desconto?", desconto)
12
13 # Usando NOT
14 print("Não é estudante?", not estudante)
```



Estácio

- Misturam operadores aritméticos, relacionais e lógicos.
- Exemplo: `(media >= 7) and (faltas < 10)`
- Interpretação: aprovado se média maior ou igual a 7 e menos de 10 faltas.



- Programa que lê duas notas de um aluno.
- Calcula a média.
- Exibe:
 - “Aprovado” se média ≥ 7 .
 - “Recuperação” se média entre 5 e 6.9.
 - “Reprovado” se média < 5 .



- Criar programa que:
 - 1 Pergunta idade e se possui carteira de habilitação.
 - 2 Usa operadores relacionais e lógicos para decidir:
 - Se pode dirigir.
 - Se precisa esperar.
 - 3 Mostrar mensagem apropriada.



- Aprendemos operadores aritméticos, relacionais e lógicos.
- Vimos como combinar operadores em expressões.
- Exercícios reforçaram lógica condicional.
- Próxima aula: **Python III — Estruturas Condicionais.**



- Compreender o funcionamento das estruturas condicionais em Python.
- Aprender a usar o comando `if`, `else` e `elif`.
- Resolver problemas que exigem decisões múltiplas.
- Exercitar raciocínio lógico em exemplos práticos.



- Revisão: operadores relacionais e lógicos.
- Estruturas condicionais simples e compostas.
- Estruturas aninhadas.
- Exemplos práticos em Python.
- Atividade prática em laboratório.
- Exercício em grupo.



Estrutura Condicional Simples

```
1 idade = 18
2
3 if idade >= 18:
4     print("Maior de idade")
```

- Executa apenas se a condição for verdadeira.
- Caso contrário, nada acontece.



Estácio

Estrutura Condicional Composta

```
1 nota = 6.5
2
3 if nota >= 7:
4     print("Aprovado")
5 else:
6     print("Reprovado")
```

- Há sempre dois caminhos: verdadeiro ou falso.



Estácio

Estrutura Condicional Encadeada

```
1 nota = 5.8
2
3 if nota >= 7:
4     print("Aprovado")
5 elif nota >= 5:
6     print("Recuperação")
7 else:
8     print("Reprovado")
```

- Permite várias verificações em sequência.



Estácio

Estruturas Aninhadas

```
1 idade = 20
2 tem_carteira = True
3
4 if idade >= 18:
5     if tem_carteira:
6         print("Pode dirigir")
7     else:
8         print("Precisa tirar a carteira")
9 else:
10    print("Ainda não pode dirigir")
```



Estácio

Exemplo Prático — Calculadora Simples

```
1 num1 = int(input("Digite o primeiro número: "))
2 num2 = int(input("Digite o segundo número: "))
3 operacao = input("Digite +, -, * ou /: ")
4
5 if operacao == "+":
6     print("Resultado:", num1 + num2)
7 elif operacao == "-":
8     print("Resultado:", num1 - num2)
9 elif operacao == "*":
10    print("Resultado:", num1 * num2)
11 elif operacao == "/":
12    print("Resultado:", num1 / num2)
13 else:
14    print("Operação inválida")
```



Estácio

- Criar programa que leia a idade de uma pessoa e exiba:
 - “Criança” se idade < 12 .
 - “Adolescente” se entre 12 e 17.
 - “Adulto” se entre 18 e 59.
 - “Idoso” se 60 ou mais.



- Em duplas, criem programa que leia três notas de um aluno.
- Calcule a média.
- Exiba:
 - “Aprovado” se média ≥ 7 .
 - “Recuperação” se média entre 5 e 6.9.
 - “Reprovado” se média < 5 .
- Cada grupo deve comparar os resultados com diferentes entradas.



- Estruturas condicionais permitem decisões no programa.
- Vimos `if`, `else`, `elif` e condicionais aninhados.
- Exercícios mostraram uso em classificações e cálculos.
- Próxima aula: **Python IV — Estruturas de Repetição.**



- Entender a necessidade de repetições na programação.
- Utilizar laços `while` e `for` em Python.
- Diferenciar quando usar cada tipo de laço.
- Resolver problemas práticos com estruturas de repetição.



- Motivação: por que repetir instruções?
- Laço `while`.
- Laço `for`.
- Função `range()`.
- Exemplos práticos.
- Atividades individuais e em grupo.



Por que usar Repetições?

- Evitam duplicação de código.
- Permitem automatizar tarefas repetitivas.
- Tornam o programa mais legível e eficiente.



Estácio

Laço While

```
1 # Executa enquanto a condição for verdadeira
2 contador = 1
3 while contador <= 5:
4     print("Contagem:", contador)
5     contador += 1
```

- Usado quando não sabemos exatamente quantas vezes repetir.
- Exemplo típico: repetir até que o usuário digite “sair”.



Estácio

Laço For

```
1 # Percorre um intervalo de valores
2 for i in range(1, 6):
3     print("Contagem:", i)
```

- Usado quando sabemos quantas vezes repetir.
- Itera sobre sequências: listas, strings, intervalos.



Estácio

A Função Range()

```
1 # range(início, fim, passo)
2
3 for i in range(0, 10, 2):
4     print(i)
5
6 # Saída: 0, 2, 4, 6, 8
```

- Intervalo de valores.
- Parâmetros: início, fim (não incluído), passo.



Estácio

Exemplo Prático — Tabuada

```
1 numero = int(input("Digite um número: "))
2
3 print("Tabuada do", numero)
4 for i in range(1, 11):
5     print(numero, "x", i, "=", numero * i)
```



Estácio

Exemplo Prático — Soma de Números

```
1 soma = 0
2 n = int(input("Digite um número (0 para parar): "))
3
4 while n != 0:
5     soma += n
6     n = int(input("Digite um número (0 para parar): "))
7
8 print("Soma total:", soma)
```



Estácio

- Criar programa que leia um número e exiba a contagem regressiva até 0.
- Exemplo: entrada = 5 $\rightarrow$ saída: 5, 4, 3, 2, 1, 0.
- Desafio extra: ao final, mostrar “Fogo!”.



- Criar jogo de adivinhação:
 - 1 O programa escolhe um número secreto entre 1 e 20.
 - 2 O jogador tenta adivinhar.
 - 3 Enquanto não acertar → programa dá dicas: “maior” ou “menor”.
 - 4 Contar quantas tentativas foram necessárias.



- Repetições tornam programas mais eficientes.
- `while`: repete enquanto condição verdadeira.
- `for`: repete em intervalos ou coleções.
- Exercícios mostraram aplicações em cálculos e jogos.
- Próxima aula: **Python V — Listas e Estruturas de Dados Simples.**



- Compreender o conceito de listas em Python.
- Criar e manipular listas.
- Acessar elementos por índice.
- Usar operações básicas: adicionar, remover, percorrer.
- Resolver problemas práticos com listas.



- O que são listas?
- Criando listas.
- Acessando elementos.
- Operações comuns.
- Laços com listas.
- Atividade prática e exercício em grupo.



O que são Listas?

- Estrutura de dados que armazena múltiplos valores.
- Os elementos são **ordenados** e acessados por índice.
- Suportam diferentes tipos: números, strings, até listas dentro de listas.



Estácio

Criando Listas

```
1 # Lista de números
2 numeros = [10, 20, 30, 40]
3
4 # Lista de strings
5 nomes = ["Ana", "Roni", "Carlos"]
6
7 # Lista mista
8 mistura = [1, "texto", True, 3.14]
```



Estácio

Acessando Elementos

```
1 frutas = ["maçã", "banana", "uva", "laranja"]  
2  
3 print(frutas[0])    # maçã  
4 print(frutas[2])    # uva  
5 print(frutas[-1])   # laranja (último elemento)
```



Estácio

```
1 frutas = ["maçã", "banana", "uva"]
2
3 frutas.append("laranja")      # adiciona
4 frutas.remove("banana")      # remove
5 print(len(frutas))           # tamanho da lista
6 print("uva" in frutas)       # verifica se existe
```



Percorrendo Listas

```
1 numeros = [1, 2, 3, 4, 5]
2
3 for n in numeros:
4     print("Número:", n)
5
6 # Soma dos elementos
7 soma = 0
8 for n in numeros:
9     soma += n
10 print("Soma total:", soma)
```



Estácio

Exemplo Prático — Lista de Alunos

```
1 alunos = []  
2  
3 for i in range(3):  
4     nome = input("Digite o nome do aluno: ")  
5     alunos.append(nome)  
6  
7 print("Lista final de alunos:", alunos)
```



Estácio

- Criar programa que:
 - 1 Leia 5 números inteiros digitados pelo usuário.
 - 2 Armazene em uma lista.
 - 3 Exiba a lista completa.
 - 4 Mostre o maior e o menor número.



- Em duplas, criar programa que:
 - 1 Solicite nomes de alunos até digitar “fim”.
 - 2 Armazene os nomes em uma lista.
 - 3 Ao final, mostre:
 - Quantos alunos foram cadastrados.
 - A lista em ordem alfabética.



- Listas permitem armazenar vários valores em uma única variável.
- Aprenderemos a criar, acessar e manipular elementos.
- Laços facilitam operações sobre listas.
- Próxima aula: **Python VI — Funções**.



- Entender o conceito de funções em programação.
- Criar funções em Python com e sem parâmetros.
- Utilizar retorno de valores.
- Praticar modularização e reutilização de código.



- O que são funções?
- Vantagens da modularização.
- Criando funções em Python.
- Parâmetros e retorno.
- Exemplos práticos.
- Atividade individual e em grupo.



O que são Funções?

- Blocos de código reutilizáveis.
- Executam uma tarefa específica.
- Podem receber dados (parâmetros) e devolver resultados (retorno).



Estácio

Estrutura Básica de uma Função

```
1 def nome_da_funcao():  
2     # código da função  
3     print("Executando a função")  
4  
5 # Chamando a função  
6 nome_da_funcao()
```



Estácio

Funções com Parâmetros

```
1 def saudacao(nome):  
2     print("Olá, ", nome, "!")  
3  
4 saudacao("Roni")  
5 saudacao("Ana")
```

- Permitem personalizar a execução.



Estácio

Funções com Retorno

```
1 def quadrado(x):  
2     return x * x  
3  
4 resultado = quadrado(5)  
5 print("O quadrado é:", resultado)
```

- O comando `return` devolve um valor para quem chamou a função.



Estácio

Exemplo Prático — Calculadora com Funções

```
1 def soma(a, b):  
2     return a + b  
3  
4 def subtracao(a, b):  
5     return a - b  
6  
7 def multiplicacao(a, b):  
8     return a * b  
9  
10 def divisao(a, b):  
11     return a / b  
12  
13 print("Soma:", soma(3, 2))  
14 print("Multiplicação:", multiplicacao(4, 5))
```



Estácio

- Criar função que receba uma lista de números e devolva:
 - O maior número.
 - O menor número.
 - A média da lista.
- Testar a função com diferentes listas.



- Em trios, desenvolver programa que:
 - 1 Tenha uma função para calcular IMC.
 - 2 Tenha uma função que classifique o resultado:
 - Abaixo do peso, Normal, Sobrepeso, Obesidade.
 - 3 No programa principal, pedir peso e altura de 3 pessoas.
 - 4 Mostrar o resultado para cada uma.



- Funções tornam programas mais organizados.
- Podem receber parâmetros e retornar valores.
- Foram usadas para cálculos e classificação de dados.
- Próxima aula: **Python VII — Projeto Integrador em Grupos.**



- Integrar os conteúdos aprendidos até agora em um projeto prático.
- Desenvolver habilidades de trabalho em equipe e resolução de problemas.
- Aplicar lógica, variáveis, estruturas condicionais, loops, listas e funções.
- Estimular a criatividade e a autonomia dos grupos.



- Revisão rápida dos conceitos principais.
- Apresentação da proposta do projeto.
- Formação dos grupos.
- Desenvolvimento prático em laboratório.
- Apresentação parcial dos resultados.



- Variáveis e tipos de dados.
- Operadores e expressões.
- Estruturas condicionais (if, elif, else).
- Estruturas de repetição (for, while).
- Listas e manipulação de dados.
- Funções para modularização do código.



- Criar um programa em Python que seja:
 - 1 Um **jogo simples** (quiz, adivinhação, batalha de números, etc.); **ou**
 - 2 Um **sistema utilitário** (calculadora, agenda de contatos, controle de tarefas, etc.).
- Requisitos mínimos:
 - Uso de listas.
 - Uso de pelo menos uma função definida pelo grupo.
 - Estruturas condicionais e laços.
 - Entrada e saída de dados.



Exemplo de Ideia — Quiz Interativo

```
1 perguntas = ["Capital do Brasil?", "2 + 2 = ?"]
2 respostas = ["Brasilia", "4"]
3 pontos = 0
4
5 for i in range(len(perguntas)):
6     resp = input(perguntas[i] + " ")
7     if resp == respostas[i]:
8         pontos += 1
9
10 print("Você acertou", pontos, "de", len(perguntas))
```



Estácio

Exemplo de Ideia — Agenda de Contatos

```
1 agenda = []
2
3 def adicionar(nome, telefone):
4     agenda.append((nome, telefone))
5
6 def listar():
7     for contato in agenda:
8         print("Nome:", contato[0], "- Telefone:", contato
9             [1])
10
11 adicionar("Ana", "9999-1234")
12 listar()
```



Estácio

- Divisão em grupos de 3 a 4 alunos.
- Cada grupo escolhe uma ideia de projeto.
- Desenvolvimento inicial em sala de aula.
- Professor atua como mentor, ajudando nos bloqueios.



Exercício em Grupo — Apresentação Parcial

- Ao final da aula, cada grupo deve:
 - 1 Explicar a ideia escolhida.
 - 2 Mostrar o progresso alcançado (mesmo que parcial).
 - 3 Compartilhar desafios encontrados.
- Feedback coletivo e troca de sugestões.



Estácio

- Revisamos os principais conceitos de Python já estudados.
- Iniciamos um projeto integrador em grupos.
- Os alunos puderam exercitar criatividade e colaboração.
- Próxima aula: **Aula 18 — Revisão e Preparação para o Projeto Final.**



- Revisar os conceitos principais do curso.
- Reforçar a integração entre teoria e prática.
- Preparar os alunos para o desenvolvimento do Projeto Final.
- Orientar quanto a requisitos, prazos e critérios de avaliação.



- Revisão geral do conteúdo.
- Discussão sobre principais dificuldades encontradas.
- Apresentação do Projeto Final.
- Formação e organização dos grupos.
- Definição de cronograma de entrega.



- Pensamento Computacional: decomposição, padrões, abstração, algoritmos.
- Scratch: blocos, estruturas de controle, variáveis, mini-jogos.
- Python:
 - Entrada e saída de dados.
 - Operadores e expressões.
 - Condicionais.
 - Estruturas de repetição.
 - Listas.
 - Funções.



- O que foi mais fácil de aprender?
- Quais foram os maiores desafios?
- Como os conceitos podem ser aplicados fora da disciplina?
- Espaço aberto para dúvidas antes do projeto final.



Apresentação do Projeto Final

- Desenvolver um programa em Python que:
 - ① Resolva um problema real do cotidiano **ou**
 - ② Represente um jogo interativo simples.
- Requisitos obrigatórios:
 - Uso de variáveis, condicionais e laços.
 - Uso de listas para armazenar informações.
 - Definição de pelo menos duas funções.
 - Interação com o usuário (entrada e saída).



Estácio

Exemplos de Projetos Possíveis

- Quiz com perguntas e pontuação.
- Agenda de contatos com busca e remoção.
- Sistema simples de biblioteca (cadastro e empréstimo).
- Jogo da forca.
- Simulação de caixa eletrônico.



Estácio

- Grupos de 3 a 4 alunos.
- Papel de cada integrante:
 - Programador principal.
 - Auxiliar de lógica.
 - Documentador (explicação do código).
 - Testador.
- Todos devem entender o código final.



Cronograma do Projeto

- Aula 18: definição de ideias e grupos.
- Aula 19: desenvolvimento e acompanhamento com professor.
- Aula 20: apresentação final e entrega.



Estácio

- Revisamos os principais conceitos da disciplina.
- Identificamos dúvidas e desafios.
- Apresentamos as diretrizes do Projeto Final.
- Próxima aula: **Aula 19 — Desenvolvimento do Projeto Final.**



- Dedicar a aula inteira ao desenvolvimento do Projeto Final.
- Aplicar os conceitos trabalhados ao longo da disciplina.
- Estimular trabalho em equipe e divisão de responsabilidades.
- Contar com a mediação do professor como mentor.



- Organização inicial dos grupos.
- Definição detalhada da ideia escolhida.
- Implementação do código em Python.
- Registro das etapas (documentação simples).
- Feedback contínuo do professor.



- Lembrar funções de cada integrante:
 - Programador principal.
 - Auxiliar de lógica.
 - Documentador.
 - Testador.
- Divisão equilibrada das tarefas.



Etapas do Desenvolvimento

- 1 Definir escopo mínimo do projeto.
- 2 Planejar quais funções serão necessárias.
- 3 Implementar os primeiros módulos.
- 4 Testar cada parte antes de integrar.
- 5 Registrar problemas e soluções encontradas.



Estácio

- Circular entre os grupos para identificar dúvidas.
- Incentivar autonomia, mas orientar quando necessário.
- Estimular boas práticas:
 - Comentários no código.
 - Divisão em funções.
 - Testes progressivos.



Checklist de Qualidade do Projeto

- O programa possui interação com o usuário?
- Utiliza variáveis, condicionais e laços?
- Inclui listas e pelo menos duas funções próprias?
- É executável sem erros?
- Está devidamente explicado pelos integrantes?



Estácio

- Cada grupo deve manter um registro simples:
 - Objetivo do projeto.
 - Principais funções implementadas.
 - Problemas encontrados e soluções.
 - Integrantes responsáveis.
- Esse material será útil na apresentação final.



Encerramento da Aula 19

- Grupos devem ter versão funcional parcial do projeto.
- Preparar para finalizar ajustes na próxima aula.
- Trazer dúvidas específicas para discussão final.
- Próxima aula: **Aula 20 — Apresentação do Projeto Final.**



Estácio

- Concluir a disciplina com a apresentação dos projetos finais.
- Estimular comunicação, argumentação e espírito de equipe.
- Avaliar os projetos segundo os critérios definidos.
- Compartilhar aprendizados e celebrar os resultados.



- Preparação final dos grupos.
- Apresentações dos projetos.
- Feedback dos colegas e professor.
- Encerramento e reflexões finais.



Organização da Apresentação

- Cada grupo terá de 8 a 10 minutos:
 - 1 Explicar o objetivo do projeto.
 - 2 Mostrar o funcionamento (demonstração ao vivo).
 - 3 Destacar as principais funções usadas.
 - 4 Comentar dificuldades e aprendizados.
- Tempo adicional de 2 a 3 minutos para perguntas da turma.



Estácio

- **Funcionalidade (30%):** o programa funciona sem erros.
- **Aplicação dos conceitos (30%):** uso de variáveis, condicionais, laços, listas e funções.
- **Criatividade e relevância (20%):** originalidade da ideia e aplicabilidade.
- **Trabalho em equipe e clareza (20%):** divisão de papéis, explicação e apresentação.



- Cada grupo recebe feedback do professor.
- Breve rodada de comentários dos colegas.
- Autoavaliação:
 - O que cada integrante aprendeu?
 - Como contribuiu para o grupo?
 - O que faria diferente em um próximo projeto?



- O pensamento computacional vai além da programação.
- Habilidades desenvolvidas:
 - Resolução de problemas.
 - Lógica e raciocínio crítico.
 - Colaboração em grupo.
 - Criatividade.
- Aplicações possíveis em diversas áreas.



- Agradecimento pelo empenho e dedicação.
- Destaque para os projetos apresentados.
- Incentivo para continuar explorando Python e Ciência da Computação.
- Sugestão de próximos passos: cursos, projetos pessoais, hackathons.

